

Received October 19, 2020, accepted November 20, 2020, date of publication December 2, 2020, date of current version December 14, 2020.

Digital Object Identifier 10.1109/ACCESS.2020.3041710

A Blockchain-Based Authentication and Key Agreement (AKA) Protocol for 5G Networks

MAEDE HOJJATI¹, ALIREZA SHAFIEINEJAD¹,
AND HALIM YANIKOMEROGLU², (Fellow, IEEE)

¹Department of Electrical and Computer Engineering, Tarbiat Modares University, Tehran 14117-13116, Iran

²Department of System and Computer Engineering, Carleton University, Ottawa, ON K1S 5B6, Canada

Corresponding author: Alireza Shafieinejad (shafieinejad@modares.ac.ir)

ABSTRACT Subscriber authentication is a primitive operation in mobile networks required by each operator prior to offering any service to end users. In this paper, we propose a novel blockchain-based Authentication and Key Agreement (AKA) protocol for roaming services in 5G networks. Each Home Network (HN) creates its own smart contract and publishes its address to inform other operators who want to offer roaming services to HN subscribers. All subsequent communication between the HN and Serving Network (SN) is done by calling the function of this smart contract. The proposed protocol eliminates the need for a secure channel between the HN and SN, which is a primary requirement of current 5G AKA protocols. In practice, a secure channel requires the HN and SN to establish a secure session before running the AKA protocol. Further, the proposed protocol leverages the benefits of blockchain, such as auditable log, decentralized architecture, and the prevention of Denial of Service (DoS) attacks. Furthermore, we provide a security proof of the protocol through formal verification using ProVerif. The results show that our scheme tends to preserve user privacy and at the same time provides mutual authentication of the participants. Finally, our evaluation of the Ethereum blockchain shows that the protocol is efficient in terms of both transaction and execution costs.

INDEX TERMS 5G networks, authentication and key agreement protocol, blockchain system, formal verification.

I. INTRODUCTION

Recent developments in the mobile network industry provide new services and applications. More specifically, developments in 5G networks present a set of new services, including Ultra-Reliable Low Latency Communications (URLLC), Massive Machine-Type Communications (MMTC), and Enhanced Mobile Broadband (EMB). Regarding these new services, several challenges like decentralization, transparency, interoperability, privacy, and security have emerged in the process of their implementation.

Among these challenges, security is one of the most important issues, and it imposes specific constraints on developers of 5G networks to make designs compatible with new technologies. Security issues originate with data sensitive applications, such as banking, healthcare, mobile cloud, etc. In these scenarios, it is critical for the services to authenticate each customer correctly as well as to preserve their privacy.

The associate editor coordinating the review of this manuscript and approving it for publication was Xiaodong Xu¹.

Broadcasting, which is a distinct feature of wireless transmissions, is a major source of threats. Some security vulnerabilities include eavesdropping, traffic analysis, jamming, Man In The Middle (MITM), Denial of Service (DoS) and Distributed Denial of Service (DDoS) [1].

The 3rd Generation Partnership Project (3GPP) group, as the developer of the standards for 3G, 4G, and 5G, has paid particular attention to security issues, such as subscriber privacy and the protection of sensitive data. The 3GPP has published a series of documents on security architecture and procedures of 5G systems. The latest version, v15.4.0 of Release 15 of the Technical Specifications (TS), was published in May 2019 (TS 33.501) [2]. The issues discussed included security requirements and features for User Equipment (UE), Home Networks (HNs) and Serving Networks (SNs) for authentication, information leakage, user localization, privacy protection and effects of the active/passive attackers.

Further, TS 33.501 drew attention to a case of a malicious SN while the user gets roaming service. Roaming is an

important service in mobile communication, when the UE enters into a cell out of the HN coverage area. Roaming is expected to become more prominent in 5G networks due to the popularity of both local 5G networks and micro 5G operators [3], [4]. Since both have lightweight implementation, they are more vulnerable to threats from active attackers and thereby can act as malicious serving networks.

Further, a large section of TS 33.501 focused on the authentication of participants, including UE, HNs, and SNs. The goal was to establish a secure channel between the UE and SN and authenticate them with the HN. The 3GPP has specified three authentication methods: 5G AKA, EAP-AKA and EAP-TLS. A suitable option is picked up by the HN whenever it has correctly identified the subscriber with an Initialization Protocol. The 5G AKA protocol is the improved version of the AKA protocol currently used in 4G (identified as EPS-AKA) which is assumed to provide required security guarantees. The introduction of AKA by the 3GPP has motivated researchers to improve the proposed authentication protocols. In this paper, we discuss the following set of issues:

Deploying blockchain for 5G authentication processes: Blockchain technology provides a platform for an immutable distributed ledger. The second generation of this technology enables the execution of programmable transactions in the form of smart contracts [5]. Blockchain for 5G authentication can serve as a barrier between the SN and HN. In so doing, blockchain can provide a secure channel for exchanging messages. This promotes user anonymity and protects the HN from DoS attacks by making it unreachable by malicious SNs, which can act as active attackers. It also provides a tamper-proof and auditable log of the authentication processes.

The vulnerabilities of the current 5G AKA variants: We discuss the vulnerabilities of the basic 5G AKA protocol and the improved state-of-the-art versions.

Eliminating the need for a secure channel in 5G AKA: A common requirement of the current 5G AKA protocol, and its preceding variants, is that of needing a secure channel between the HN and SN. In fact, this is a vital condition for the security proof of current protocols. However, this condition is restrictive because it requires the channel to be private and thus inaccessible to any passive or active attacker. In practice, a secure channel imposes another authentication and key exchange protocol to the mobile networks. This protocol is run between the HN and SN which communicate with each other through an IP network. This is achieved by the Internet Key Exchange (IKE) and IPSec protocols which are based on either pre-shared keys or digital certificates [6]. In this paper, we present a method of eliminating this requirement and providing a strong security proof as well as simplifying the deployment of 5G AKA protocol.

Security evaluation of 5G AKA. We carry out a formal security evaluation of the proposed authentication protocol and provide a comprehensive analysis.

A. RELATED WORK

The introduction of AKA by 3GPP triggered enough motivation for researchers to improve the proposed authentication protocols [2]. The authors in [7], revealed a new privacy attack against all variants of the AKA protocol, including 5G AKA. This attack compromised subscriber privacy more severely than any known location privacy attacks. Further, the authors in [7] conducted a security analysis of the corresponding vulnerability. Liyanage *et al.* identified the most important privacy issues caused by the new technologies, which are expected to be utilized in 5G [8].

Basin *et al.* followed the research on AKA by providing a formal analysis of the protocol in order to identify the missing security goals [9]. They used *Tamarin* [10], a security protocol verification tool for formal verification. In addition to detecting security goals missed in the primary design, they made recommendations to fix the corresponding flaws and weaknesses.

More recently, Braeken *et al.* proposed a new version of the 5G AKA to overcome the identified weaknesses in the protocol [11]. The main idea was to replace the sequence number with random numbers and thereby reduce the number of required communication phases in the protocol. Further, they claimed that the proposed protocol achieved two extra security goals that were missed in 5G AKA, namely post compromise security and forward security.

From architecture point of view, blockchain-based solutions in 5G networks are in the initial state-of-the-art. Recently, several technologies, such as Software-Defined Networking (SDN), Network Function Virtualization (NFV), machine learning, and cloud computing are used in 5G networks to improve the quality of services. But this leads to several challenges, like decentralization, transparency, interoperability, privacy, and security. Blockchain is capable of addressing these issues since it has inherent features such as transparency, data encryption, auditability, immutability and distributed architecture [12]. From a security point of view, blockchain has the potential to provide solutions for data privacy, authentication, integrity protection, and access control.

Refaey *et al.* studied the benefits and drawbacks of blockchain in roaming systems [13]. They proposed a high-level architecture for a roaming model and discussed potential issues and challenges. The model was evaluated using the case of roaming in the EU.

The authors in [14] proposed a blockchain framework for trusted task sharing in Multi-access Edge Computing (MEC) environments. The framework was deployed on the Hyperledger blockchain and its performance was evaluated from the point of view of latency.

The authors in [15] proposed a blockchain-based security authentication scheme for 5G Ultra-Dense Networks (UDNs). UDNs are a technology meant to address the issue of network system capacity in 5G networks. They are composed of a set of access points (APs) with a smaller coverage

area than a base station, which is primarily characterized by being autonomous, temporary, and dynamic. The scheme is mainly focused on the interaction between the UE and APs by defining a trusted APs group (APG). The work proposed an APG-PBFT algorithm based on blockchain with a Byzantine fault tolerance (PBFT) consensus algorithm. The trusted APG was generated using this algorithm and the authentication results could be shared in the APG with the propagation mechanism.

From an experimental point of view, Morozo *et al.* designed and implemented an intra-operator charging framework model for roaming based on blockchain [16]. The MNOs were integrated into a blockchain consortium as full nodes. The proposed model allows for the verification of transactions by the miners while processing smart contracts.

B. CONTRIBUTIONS OF THE WORK

In this paper, we propose an authentication protocol for 5G networks based on blockchain. Blockchain can serve as a barrier between the HN and SN and can provide a public channel for message exchanging. This model is appropriate for roaming, where UE uses a different network than its home network. Given the significant number of customers, mobile operators will lose many of their original customers within few years if they do not understand the reasons for customer churn over time [17]. Accepting new subscribers from other operators will also be costly for both parties. Thus, it is better for an operator to retain their customers and take on the roaming service [18].

In addition to the heavy costs for both operators, subscriber migration causes customer dissatisfaction [19]. Providing services to subscribers of other operators in such a way that each operator retains its subscribers will provide satisfaction for all parties.

Operators have always tried to satisfy their customers through mutual agreements and by cooperating with each other. They also provide many services to each other's subscribers according to pre-defined agreements. Prior to using any service offered by the visited network, the subscriber must be authenticated by the HN. The authentication not only guarantees the registration of the subscriber to the HN, but also allows the visited network to retrieve the required information in order to create a session for the subscriber.

In the proposed protocol, each HN creates its own smart contract and publishes the address of this smart contract in order to inform the other operators who want to provide roaming service to the HN's subscribers. All subsequent communications between the HN and SN are done by calling the function of this smart contract. The proposed protocol tends to preserve user privacy while at the same time preventing the HN from DoS attacks by keeping it unreachable to attackers or malicious SNs. It also provides non-tampered and auditable logs of the authentication process. This paper is the first work of 5G AKA with blockchain and can be considered as a basis for future works and new ideas in this field.

The main contributions of this paper can be summarized as follows:

- We propose a blockchain-based authentication protocol in 5G networks. The framework provides more security controls including protection against malicious HN, replay attacks and DoS attacks than previous AKA schemes.
- The proposed protocol eliminates a common limitation of previous protocols, which require a secure channel between the HN and SN. This feature makes the HN free to establish secure sessions with other operators. Thus, MNOs do away with conventional peer-to-peer authentication and key exchange protocol in order to create a secure channel.
- We provide the security proof of the proposed protocol by formal verification.

Blockchains have inherent benefits, such as auditability and undeniable logs. The proposed protocol can be implemented on any distributed blockchain platform, and it partakes of blockchain's benefits. DoS prevention is another important issue provided by blockchain. In our scheme, the HN is protected against DoS attacks originating from malicious SNs. Moreover, the secrecy of sensitive data is protected by encrypting all of the exchanged messages, and consequently user privacy is maintained. Message encryption prevents a blockchain node from transaction tracing aimed at extracting user information. Thus, the participant MNOs are the only nodes able to decrypt the authentication request/response messages, which are embedded in the metadata of the transactions.

Further, smart contract can verify the freshness of authentication request by means of a unique identifier derived from random numbers of UE and SN and accordingly preventing replay attacks. In addition to the above features, the proposed protocol is capable of employing digital currency, which facilitates the charging system by eliminating the dependency on national currencies. This leads to an "anytime-anywhere" service and creates a new evolution in billing systems.

The requirement of a secure channel between the HN and SN is vital for the security proof of previous protocols. In practice, this condition is very restrictive since it assumes that the channel is private and not accessible to any passive or active attacker. By using a public blockchain as a communication channel between operators, the attacker is always able to scan the blockchain transactions and retrieve the authentication messages. However, we prove that the attacker is not capable of decrypting the messages and thus is unable to implement a successful attack.

Compared with [2], our protocol design is based on the latest version of 5G AKA, in which security holes have been fixed during various releases. Further, we verify the proposed model with formal methods in order to provide proof of the required security features. Compared to [9] and [11], our work is based on blockchain.

The rest of this paper is outlined as follows. Section II provides a brief background of current protocols for

5G authentication. Section III presents our proposed protocol and explains its phases. Section IV offers proof of the security of the proposed protocol using ProVerif, and an evaluation of the protocol's performance is provided in Section V. We discuss the application of this scheme in aerial networks. Finally, Section VII concludes the paper.

II. PRELIMINARIES

A. NOTATIONS

Table 1 summarizes the definitions and symbols used throughout the paper. Note that f_1 , *Keyseed*, and *challenge* are the primitive one-way keyed cryptographic functions which are embedded in the USIM¹ card [20].

TABLE 1. Notations used in protocol modeling.

Symbols	Description
SC	Smart Contract
ID_U	Identifier of participant U
R_i	Random number
SN	Serving Network
HN	Home Network
UE	User Equipment
Res	User challenge Response
$hxRes$	Hash form of expected Response
$xMac$	Expected Message Authentication Code
req_id	Unique identifier for authentication request
res_id	Unique identifier for authentication response
PK_U	Public Key of the principal U
SK_U	Private Key of the principal U
$SUPI$	Subscription Permanent Identifier
$SUCI$	Subscription Concealed Identifier
$h(\cdot)$	Hash function
$f_1(K, \dots)$	Primitive keyed one-way function
$keyseed_K(\dots)$	Primitive keyed one-way function used for generation of the session key
$challenge_K(\dots)$	Primitive keyed one-way function used for generation of the output
$E_K(m)$	Asymmetric encryption of message m with public key K
$D_K(c)$	Asymmetric decryption of ciphertext c with private key K
$E_K(m)$	Symmetric encryption of m with key K
$D_K(c)$	Symmetric decryption of c with key K

It is important to note that f_1 is used as message authentication function while the others are used as key generating function. The key is the USIM internal key which is shared by the HN. To our best knowledge, the inner structure of these functions is not publicly known. Further, they are operator-dependent, whereby each operator creates and customizes its own primitive functions.

B. 5G AKA PROTOCOL

In this section, we provide a brief description of the latest version of AKA protocol, i.e., 15.4.0 [2]. We ignore the details about message encryption/decryption and mainly focus on the message sequences. The participants of the protocol, which are shown in Fig. 1, are composed of the following entities:

- The User Equipment (UE)
- The Home Network (HN)
- The Serving Network (SN)

¹Subscriber Identity Module



FIGURE 1. The 5G AKA architecture.

By UE, we mean smart phones and IoT devices, which are carried by a subscriber. Each UE has a universal integrated circuit card (UICC). This card can host a universal subscriber identity module (USIM) application, which securely stores a pre-shared key with the subscriber's HN. For each USIM, the UE has a unique identifier known as a Subscription Permanent Identifier ($SUPI$), which is stored in the Universal Subscriber Identity Module (USIM). $SUPI$ in 5G corresponds to the International Mobile Subscriber Identity (IMSI) in earlier protocol standards, like 4G-LTE (Long Term Evolution).

The HN is the Mobile Network Operator (MNO) who registered the UE subscriber. The SN refers to the MNO where the UE enters into its cell area to get roaming service.

Fig. 2 shows the details of the architecture according to 5G standards. Since a 5G core network has a service-based architecture (SBA), new entities have been defined. Some of the new entities relevant to 5G authentication are the following:

- SEAF
- AUSF
- UDM
- ARPF
- SIDF

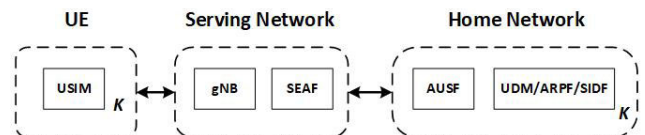


FIGURE 2. The 5G entities involved in the authentication process.

We can see that the SN is composed of two elements: gNB and SEAF. The former, Next Generation NodeB (gNB), is a base station that provides radio access network for the UE. The latter, Security Anchor Function (SEAF), is an element which acts as a mediator between the UE and its HN during the authentication process. It mainly relies on the UE's HN to accept or reject the authentication.

The other entities are located on the HN side. The Authentication Server Function (AUSF) authenticates a subscriber and makes decisions regarding UE authentication based on the backend service for computing the authentication data and key code materials. Unified data management (UDM) is an entity that supplies the main functions corresponding to data management, such as ARPF and SIDF. The Authentication Credential Repository and Processing Function (ARPF) chooses an authentication method among the three available options: 5G AKA, EAP-AKA, and EAP-TLS. The option is selected on the basis of the subscriber identity and

configured policy. The Subscription Identifier De-concealing Function (SIDF) is responsible for the decryption of the Subscription Concealed Identifier (SUCI) to obtain the SUPI.

The main assumptions about the communication channels of the protocol are that the UE communicates with a SN through a radio interface, and that the radio interface is assumed to be an insecure channel. Against, communication between SNs and a HN is based on Internet Protocol (IP), and this is supposed to be a secure channel. This means that the secure channel provides confidentiality, integrity, mutual authenticity, and replay protection, as explicitly mentioned in the standard [2]–[TS33.501, Sec. 5.9.3].

It is worth noting that deployments prior to 5G, such as 4G-LTE, had also assumed secure communication channels, such as IPsec between HN and SNs [21].

Fig. 3 shows the overall description of the different phases in the basic AKA protocol. The secure channel between the HN and SN allows credential parameters, such as $SUCI$, K_{SEAF} , $hxRes$, to securely transmit over this channel. The security goals of AKA can be summarized as follows:

- Authentication between subscribers and HNs
- Authentication between subscribers and SNs
- Authentication between a HN and SN
- Confidentiality on K_{SEAF} in case of passive attack
- Confidentiality on $SUPI$ in case of passive attack
- Confidentiality on SQN in case of passive attack
- Protection against anonymity and unlinkability in case of passive attacks

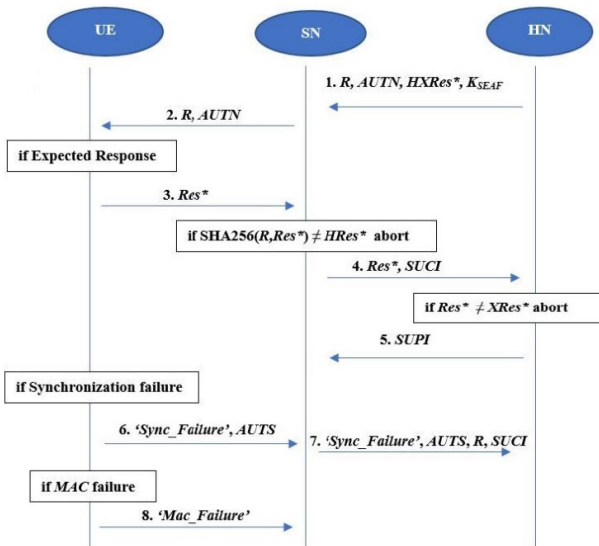


FIGURE 3. Message sequences in the 5G AKA protocol.

The main weaknesses of the current 5G AKA can be summarized as follows [11]:

- Tracking of targeted subscribers, which violates user privacy due to inadequacies, such as MAC or Synchronization.
- Information leakage from a SQN parameter, resulting in the possibility of activity monitoring attacks [7].

- Impersonation of a legitimate SN by a malicious SN, if the authentication is not fully verified by the UE. Basically, this originates from the fact that the standard does not specify additional key confirmation rounds, nor does it specify that the subscribers have to wait for this. This vulnerability has two situations specified in the standard. First, SNs are able to initiate key changes on the fly. Second, they are able to switch security contexts, including keys and parameters [9].
- An attack compromising an SN can lead to a complete breach of user privacy. This enables the malicious SN to collect $SUCI$ authentication requests from the UE and then capture the Res parameter sent by the UE over an insecure channel at a later stage. Combining the data ($SUCI$, Res) and sending it to the HN results in the retrieval of the $SUPI$.

C. BRAEKEN'S PROTOCOL

Fig. 4 shows a high-level description of the Braeken's protocol. Much like the basic AKA protocol, the secure channel is assumed between the HN and SN to allow the sensitive parameters, such as $SUCI$, K_{SEAF} , $hxRes$, to securely transmit over this interface. The improvements of Braeken's protocol over basic 5G-AKA are the following:

- The $SQNs$ are replaced by random numbers, and the USIM is required to generate strong random numbers.
- A mechanism allowing the UE to verify the validity of the authentication response that comes from both the SN and the HN (not only the HN, as in basic AKA protocol).

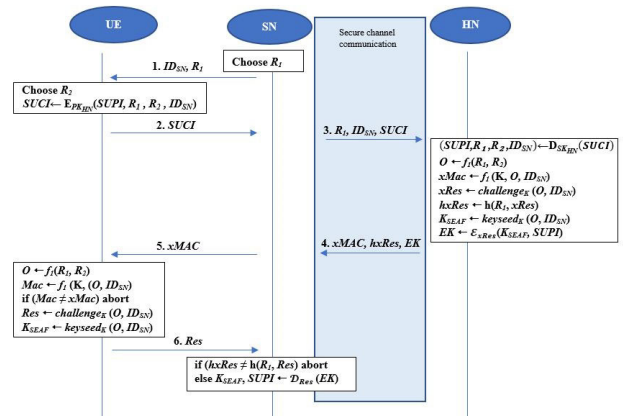


FIGURE 4. Message sequences in Braeken's protocol.

The former addresses the first two weaknesses of the AKA protocol, while the latter eliminates the last two weaknesses. Further, another contribution of the Braeken's protocol is the reduction of the required communication phases from 8 to 6.

III. PROPOSED PROTOCOL FOR 5G NETWORKS

In this section we first describe the issues regarding our design. Then we will go into further details of the proposed protocol.

A. DESIGN CONSIDERATIONS

The main ideas that have informed the design of our protocol are the following:

- We consider blockchain as an interface between HNs and other operators who want to provide roaming services to HN subscribers. Since the registration of a transaction requires the digital signature of the agent who requests the transaction, all of the authentication requests coming onto the blockchain have data origin authentication. It is important to note that both the HN and SN must have valid certificates.
- Since we assume there is an insecure channel between the HN and SN, all messages between these participants are encrypted by suitable asymmetric and symmetric encryption algorithms. In other words, the transaction metadata containing the authentication message is always encrypted. Therefore, anyone who scans the blocks of a blockchain is incapable of providing any information about user sensitive data.
- We can set up a rigid access control mechanism for calling the smart contract functions. More specifically, as the owner of the smart contract, the HN is the only node capable of registering a response transaction to authentication requests.
- We allow the HN to be involved in the key derivation algorithm by incorporating its random number in session key generation. In previous 5G AKA protocols, the session key was dependent only on the random numbers generated by the UE and the SN.
- The smart contract function responsible for the authentication request can verify the freshness of the request. In our protocol, each authentication request has a unique identifier, namely *req_id*, which allows the smart contract to abort repetitive requests and thereby preventing replay attacks.
- We employ a message sequence consisting of six rounds of message exchanges similar to Braeken's protocol in order to achieve a higher performance.

B. OVERVIEW OF THE PROTOCOL

The participants of the protocol are shown in Fig. 5. They consist of the following:

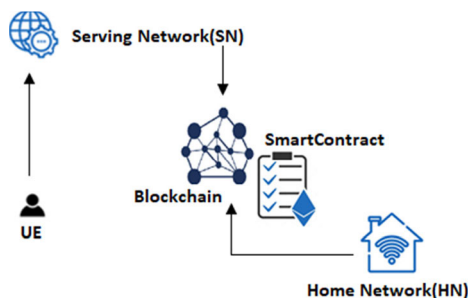


FIGURE 5. Overall architecture of the proposed protocol.

- The user who wants authentication (denoted by UE)
- The serving network (denoted by SN)
- The home network (denoted by HN)
- The smart contract of the HN (denoted by SC)

Fig. 6 shows the details of the architecture according to 5G elements. In our architecture, the SEAF is responsible for communicating with the blockchain in order to send the authentication request to the HN. The main functionality of the proposed protocol on the SN side is implemented in the SEAF unit. Thus, this module must be customized to support the blockchain-based authentication. Meanwhile, the gNB only forwards the UE messages to the SEAF unit and is not directly involved in the authentication process.

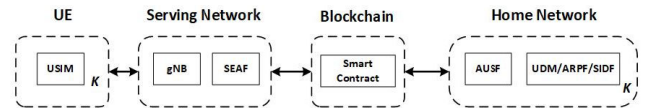


FIGURE 6. The 5G entities involved in the proposed authentication protocol.

Further, on the HN side, the AUSF module must be customized to enable blockchain-based authentication. AUSF is responsible for communicating with the blockchain to get the authentication request from SEAF and provide the response. The authentication process is done by the UDM, and the final decision is redirected to AUSF.

C. INITIALIZATION

In our proposed model, each HN creates its own smart contract on a public blockchain. The operator publicly publishes the address of the smart contract (e.g., through its official website). Each SN who wants to offer roaming service to an incoming HN's subscriber communicates via this smart contract. The use of a public blockchain frees the HN from building any infrastructure, thereby following the pay-per-service business model. Further, it provides an integration platform for the MNOs who want to offer roaming service to each other.

D. MESSAGE SEQUENCES OF THE PROTOCOL

The overall design of the protocol is depicted in the schematic diagram in Fig. 7. It consists of eight messages exchanged between participants of the protocol. Each message labeled by a number which identifies a phase of the protocol. In the following subsections, each phase is described in more details.

1) PHASE 1: INITIAL REQUEST BY THE SN

The user enters into a foreign network cell and receives an authentication request to join the SN. This step is triggered after the UE completes the Radio Resource Control (RRC) procedure with gNB and sends an Attach Request message to the Mobility Management Entities (MME). The SN generates a random number (R_1) and sends it to the user as well as to its identifier (ID_{SN}).

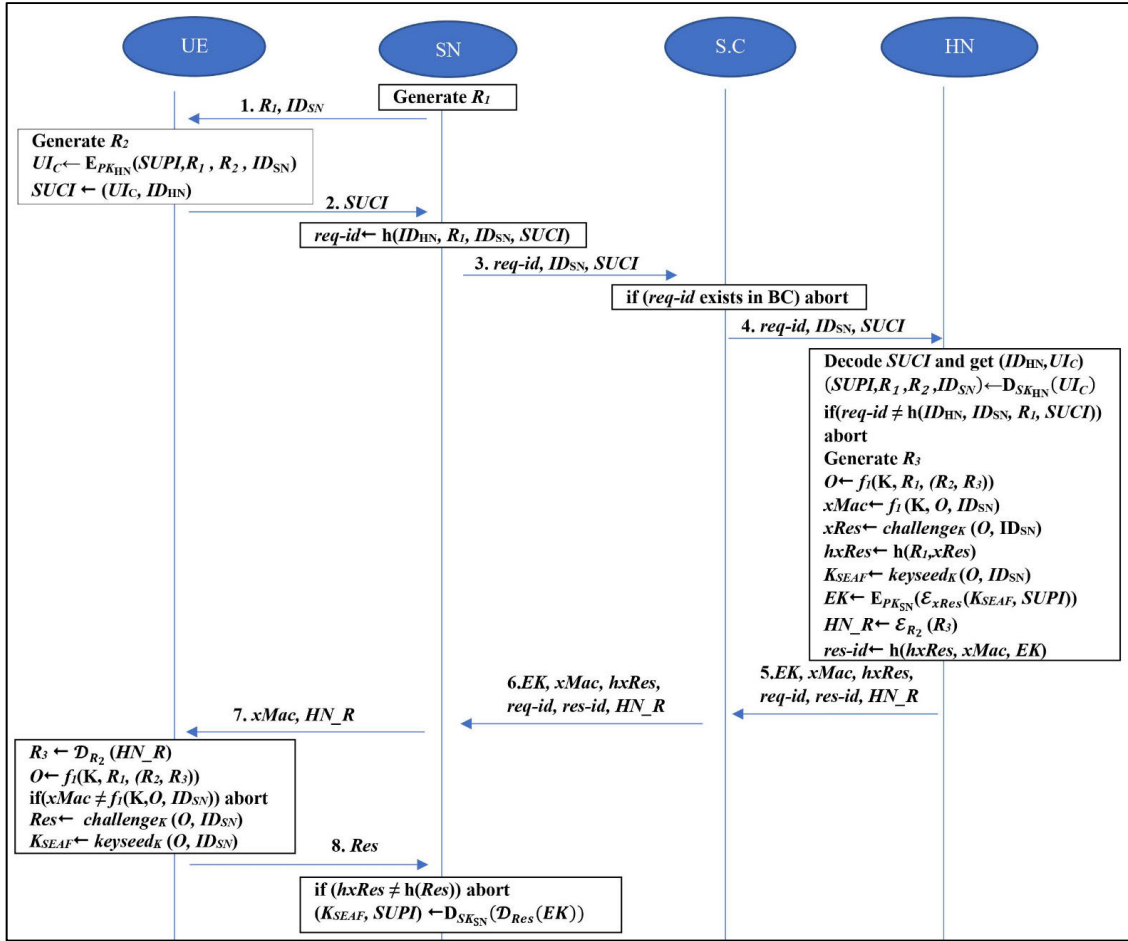


FIGURE 7. The flow of protocol messages.

2) PHASE 2: INITIAL USER RESPONSE

In this phase, the user takes the following steps to compute the $SUCI$ according to Algorithm 1:

- Generates R_2 randomly;
- Encrypts the message composed of $SUPI$, R_1 , R_2 , and ID_{SN} and, using public key encryption with the public key of the HN, puts the encrypted result into UIC ;
- Creates the $SUCI$ response message containing both UIC and its HN identifier (ID_{HN}).

Then, the user sends the $SUCI$ to the SN.

Algorithm 1 The User Response to the Challenge of the SN

Input: ID_{SN} , R_1

1. Generate random number R_2
2. $UIC \leftarrow E_{PK_{HN}}(SUPI, R_1, R_2, ID_{SN})$
3. $SUCI \leftarrow (UIC, ID_{HN})$
4. **Send** ($SUCI$) to the SN

3) PHASE 3: REGISTERING REQUEST BY THE SN

Upon receiving the user response, the SN computes the hash value of ID_{SN} , R_1 , ID_{HN} , and $SUCI$ to obtain a unique identifier for the authentication request denoted by req_id .

Finally, the SN sends the authentication request, composed of req_id , ID_{SN} , and $SUCI$ to the HN by registering a transaction request to the smart contract of the HN.

Algorithm 2 Registering Request of the SN to the Smart Contract

Input: $SUCI$

1. $req_id \leftarrow h(ID_{SN}, R_1, ID_{HN}, SUCI)$
2. The SN **registers** the transaction (req_id , ID_{SN} , $SUCI$)

4) PHASE 4: FORWARDING REQUEST TO THE HN

Upon receiving the request, the smart contract first checks the freshness of the request by seeking an authentication record with req_id in order to prevent duplicate requests. If the incoming request is determined to be a fresh request, it is redirected to the HN agent. Otherwise, the request is rejected and the transaction will be reverted. In summary, the smart contract function is responsible for prevention of replay attacks originated from malicious SNs.

5) PHASE 5: REGISTERING RESPONSE BY THE HN

Upon receiving the request from the smart contract, the HN takes the following actions:

- Checks its identifier with the ID_{HN} located in $SUCI$;
- Decrypts UI_C with its private key to obtain $SUPI$, R_1 , R_2 , and ID_{SN} ;
- Verifies the equality of req_id with the hash result of ID_{SN} , R_1 , ID_{HN} , and $SUCI$.

The authentication process is stopped if any of the above conditions are not satisfied. Otherwise, the SN continues the with following actions:

- Generates a random number R_3 ;
- Produces O by feeding R_1 , R_2 and R_3 into f_1 ;
- Calculates the $xMac$ of the message by feeding O and ID_{SN} into keyed f_1 ;
- Calculates the $xRes$ using the keyed $challenge$ function with O and ID_{SN} as inputs;
- $hxRes$ is generated as the hash result of R_1 and $xRes$;
- Computes K_{SEAF} by feeding O and ID_{SN} into the keyseed function;
- Produces EK with double encryption. First, through symmetric encryption of the session key and $SUPI$ with $xRes$; second, through asymmetric encryption of the result with a public key of the SN;
- Generates HN_R by symmetric encryption of R_3 with key R_2 ;
- Creates a unique identifier for the response message, known as a res_id , by getting the hash result of $hxRes$, $xMac$ and EK values.

Note that f_1 , $challenge$, and $keyseed$ are the primitive cryptographic functions defined on the HN and they are embedded in the USIM card.

It is worth noting that since EK is obtained through the encryption of user information with key $xRes$, user-related information is accessible to the SN only after successful authentication. Note that R_3 is encrypted by using R_2 as an encryption key because it is the only way that a subscriber can easily obtain R_3 .

Finally, the HN registers a transaction on the blockchain containing EK , $xMac$, $hxRes$, req_id , res_id , and HN_R using the corresponding smart contract function.

6) PHASE 6: REDIRECTING THE RESPONSE TO THE SN

In this step, the smart contract can check whether the message sender is the owner of the smart contract. We can put a condition that only the HN is allowed to register a response transaction to the smart contract.

7) PHASE 7: RESPONSE OF THE SN TO THE UE

The SN receives the HN response via the corresponding smart contract function. It then redirects $xMac$ and HN_R parts to the user. The SN saves the other parts to use them after it receives a response from the user.

8) PHASE 8: FINAL RESPONSE OF THE USER TO THE SN

The user takes the following action when receiving the SN response:

- Decrypting HN_R using R_2 as key to obtain R_3 ;

Algorithm 3 The HN Response to Authentication Request

Input: req_id , ID_{SN} , $SUCI$

1. **Decode** $SUCI$ to obtain UI_C and ID_{HN}
2. $(SUPI, R_1, R_2, ID_{SN}) \leftarrow D_{SK_{HN}}(UI_C)$
3. **if** ($req_id \neq h(ID_{HN}, ID_{SN}, R_1, SUCI)$) **then** abort
4. Generate R_3
5. $O \leftarrow f_1(K, R_1, (R_2, R_3))$
6. $xMac \leftarrow f_1(K, (O, ID_{SN}))$
7. $xRes \leftarrow challenge_K(O, ID_{SN})$
8. $hxRes \leftarrow h(R_1, xRes)$
9. $K_{SEAF} \leftarrow keyseed_K(O, ID_{SN})$
10. $EK \leftarrow E_{PK_{SN}}(E_{xRes}(K_{SEAF}, SUPI))$
11. $HN_R \leftarrow E_{R_2}(R_3)$
12. $res_id \leftarrow h(hxRes, xMac, EK_C)$
13. HN **registers** the transaction (EK_C , $xMac$, $hxRes$, req_id , res_id , HN_R)

- Computing O by feeding the parameters R_1 , R_2 , and R_3 into f_1 ;
- Creating Mac using keyed f_1 with the inputs O and ID_{SN} .

Then, the user checks the above Mac with the $xMac$ received from the SN. The authentication fails if the Mac does not match with $xMac$. Otherwise, the response is non-tampered and the authentication is accepted. Then the user continues with the following steps:

- Calculates the Res using the $challenge$ function with inputs O and ID_{SN} ;
- Obtains the session key using the $keyseed$ function with inputs O and ID_{SN} ;

Finally, the Res is sent back to the SN.

Algorithm 4 User Response to the SN

Input: $xMac$, HN_R

1. $R_3 \leftarrow D_{R_2}(HN_R)$
2. $O \leftarrow f_1(K, R_1, (R_2, R_3))$
3. **if** ($xMac \neq f_1(K, (O, ID_{SN}))$) **then** abort
4. $Res \leftarrow challenge_K(O, ID_{SN})$
5. $K_{SEAF} \leftarrow keyseed_K(O, ID_{SN})$
6. **Send** Res to the SN

Upon receiving the user response, the SN computes the hash of Res and compares it with $hxRes$. If the check is true, the user is authenticated by the SN. Now, the SN is able to decrypt EK , first by symmetric decryption using Res and then by asymmetric decryption of the result using its private key to obtain K_{SEAF} and $SUPI$, respectively. From now, the user can communicate with the SN using the established session key K_{SEAF} .

IV. SECURITY PROOF

A. PROVERIF

We use a formal method to prove the security of the proposed protocol. The message exchanging is modeled by

ProVerif [22], a well-known cryptographic protocol verifier. ProVerif is designed for the analysis of secrecy and authentication properties. This is particularly beneficial in the domain of computer security. Additional properties, such as privacy, traceability, and verifiability, can also be verified using ProVerif.

B. THREAT MODEL AND SECURITY REQUIREMENTS

The security goals of the proposed protocol consist of two parts: authentication of the participants and secrecy of sensitive data. As in 5G-AKA, in the authentication of the participants we seek to provide the following:

- Authentication between subscribers and HNs
- Authentication between subscribers and SNs
- Authentication between HNs and SNs

As for the secrecy of sensitive data, we investigate the following properties:

- Confidentiality of K_{SEAF} in case of active/passive attacks
- Confidentiality of $SUPI$ in case of active/passive attacks
- Confidentiality of R_2 and R_3 in case of active/passive attacks
- Confidentiality of USIM pre-shared key
- Protection against unlinkability in case of active/passive attacks

The confidentiality of K_{SEAF} and $SUPI$ are the same as 5G-AKA except that we consider both passive and active attacks. The third item considers the confidentiality of random numbers generated by the UE and the HN. Actually, these numbers replace the simple incremental counter, namely SQN , in 5G-AKA which is generated by the HN. The secrecy of $SUPI$ and USIM Internal key aims to preserve the privacy of the end user.

As for the features related to ability of the attacker, these can be summarized as follows:

- The interface between the SN and HN are considered as a public channel in addition to the radio interface between the UE and SN. This means that an attacker can eavesdrop on all messages exchanged over these channels.
- We consider an active attacker model, that is the attacker can inject a new message into a public channel or can save a message for future use. Since both the channels used in the protocol are public, the attacker can act as the HN, the SN or the UE. In practice, an active attacker could set up a fake base station to send and receive signaling messages and thereby impersonating SNs.
- The attacker can request to run many instances of the protocol to investigate the interleaving attacks.

The only limitation of the attacker is in accessing key materials. That is, the attacker has no access to the pre-shared key in USIM, the private key of the HN, or the private key of the SN. It is worth noting that in current 5G AKA modeling (like in [9] and [11]) the channel between the SN and HN is assumed to be secure, and accordingly the attacker cannot

access the messages exchanged over this channel. Further, the attacker is assumed to be passive in many threat models. This means the attacker is unable either to edit an existing message or to inject a new message into the channel.

C. PROTOCOL MODELING

The modeling consists of three main parts: declarations, process macros, and the main process, which will be described in more detail below. For the HN and SN, we consider the generation of a public and private key pair. To begin with, the key pair is used for the encryption and decryption of the message. Second, we use the key pair for message signing and verification. Note that the proposed protocol does not directly use the digital signature in its message sequences. This is implicitly done by the participants when the transaction is registered in the blockchain. More specifically, it is done either when the SN registers the authentication request or when the HN registers the authentication response.

1) DECLARATIONS

The declaration part includes three sections: the section for user-defined types, the section for the definition of objects, and the section where constructors are defined. In the first part, `privateKey` and `publicKey` indicate the private and public key pair used for the asymmetric encryption and digital signature. While `Key` and `UEKey` are the types used for the symmetric encryption key.

In the second part, the first two statements declare a couple of communication channels: *airChannel* and *bcChannel*, which are used for modeling the channel between the UE and SN and the channel between the SN and HN, respectively.

The concept of a *free* statement in ProVerif is the same as that of global scope in programming languages; that is, *free* names are known globally to all processes. The object definition is terminated with an access method identified by either `[public]` or `[private]` keywords. The objects that are not known by the attacker must be declared private, while the ones that are known by the attacker are declared public. By default, the access method is public (i.e., when it is not explicitly declared, the tool considers it as public).

The presence of *free* statements with a public access method in the definition of *airChannel* and *bcChannel* indicates that both the channels are known by the attacker as well as he/she can eavesdrop any message exchanged over them. This definition reflects the realistic assumption about the communication channel between HN and SN which is provided by the public blockchain. Since in public blockchain each desired node can join to the blockchain network and acts as a full node and accordingly has access to transaction data as well as smart contract functions.

The objects $SUPI$, K , K_{SEAF} , R_3 , and R_2 respectively denote the USIM unique identifier, the USIM internal key, the session key, the HN, and the SN random numbers. The keyword `[private]` in their definition implies that each one is private and thus cannot be accessed by the attacker.

Algorithm 5 Type, Object and Function Definitions

```

type UEKey.
type Key.
type publicKey.
type privateKey.

free bcChannel:channel. (*Public*)
free airChannel:channel. (*Public*)
free  $SUPI$ :bitstring [private].
free  $K$ :UEKey [private].
free  $K_{SEAF}$ :bitstring [private].
free  $R_3$ :bitstring [private].
free  $R_2$ :bitstring [private].

fun hash(bitstring):bitstring.
fun mKey (bitstring): Key.
fun pk (privateKey): publicKey.
fun pkiEnc (bitstring, publicKey): bitstring.
reduc forall m: bitstring, k: privateKey; pkiDec (pkiEnc
    (m, pk (k)), k) = m.
fun symEnc (bitstring, Key): bitstring.
reduc forall m: bitstring, k: Key; symDec (symEnc (m, k),
    k) = m.
fun sign (bitstring, privateKey): bitstring.
reduc forall m:bitstring, k: privateKey; checksign(sign(m, k),
    pk (k)) = m.
fun decode(bitstring): bitstring.
fun  $f_1$ (UEKey, bitstring, bitstring): bitstring.
fun challenge(UEKey, bitstring, bitstring): bitstring.
fun keyseed(UEKey, bitstring, bitstring): bitstring.

```

The third portion defines function symbols including both constructors and destructors. In ProVerif, each constructor builds required terms for modeling a particular primitive of cryptographic protocols. In our protocol, *hash*, *symEnc*, *symDec*, *pkiEnc*, *pkiDec*, *sign* and *getPK* are the constructors, which respectively model cryptographic hash functions, symmetric encryption, symmetric decryption, public-key encryption, digital signature, and returning the public key of a secret key.

The relationship between cryptographic primitives are captured by destructors which manipulate terms formed by constructors. In our protocol, *pkiDec*, *symDec* and *check_sign* are the destructors which respectively model public-key decryption, symmetric key decryption, and signature verification algorithms.

2) PROCESS MACROS

Instead of encoding the protocol in a single main process, we use sub-processes to declare interactions between each participant. Since the protocol has three participants as active players, in addition to the main process, three macros are defined: SN, UE and HN process. Each process identifies the act of the corresponding participant to the events occurred.

In our model, these processes correspond exactly to the proposed protocol given in Fig. 7 with the slight difference that some unnecessary details are omitted. Note that instead of modeling the blockchain or smart contract as a process macro, they are modeled by the communication channel *bcChannel*.

Algorithm 6 The SN Process

```

let  $SN(ID_{SN}:bitstring, ID_{HN}:bitstring, pk_{HN} : publicKey,$ 
     $pk_{SN} : publicKey, sk_{SN} : privateKey) =$ 
new  $R_1$ :bitstring;
event SNSendReqToUE;
out(airChannel, ( $R_1, ID_{SN}$ ));
in(airChannel,  $SUCI$ :bitstring);
let  $recv\_id_{HN} = decode(SUCI)$  in
if  $recv\_id_{HN} = ID_{HN}$  then event SNRecvUERes( $SUCI$ );
let  $req\_id = hash((ID_{HN}, R_1, ID_{SN}, SUCI))$  in
let  $Sign_{SN} = sign(req\_id, sk_{SN})$  in
event SNSendReqToHN( $req\_id$ );
out(bcChannel, ( $req\_id, Sign_{SN}, SUCI$ ));
in(bcChannel, ( $EK_C$ :bitstring,  $xMac$ :bitstring,
     $hxRes$ :bitstring,  $Sign_{HN}$ : bitstring,  $res\_id$ : bitstring,
     $HN\_R$ :bitstring));
let ( $=pk_{HN}, veri\_req\_id$ :bitstring,  $veri\_res\_id$ :bitstring) =
    checksign( $Sign_{HN}, pk_{HN}$ ) in

if ( $veri\_res\_id = res\_id$ ) then if ( $veri\_req\_id = req\_id$ )
    then event SNRecvHNRes( $Sign_{HN}$ );
event SNSendReq2ToUE( $xMac$ );
out(airChannel, ( $xMac, HN\_R$ ));
in(airChannel,  $Res$ :bitstring);
if  $hxRes = Res$  then event SNRecvUERes2;
let  $EK = pkiDec(EK_C, sk_{SN})$  in
let  $session\_info = symDec(EK, mKey(Res))$  in
let  $sn\_K_{SEAF} = decode(session\_info)$  in
let  $sn\_SUPI = decode(session\_info)$ .

```

The SN process is explained in Algorithm 6. The SN starts the protocol by selecting a random number R_1 and puts the pair (R_1, ID_{SN}) on the interface *airChannel*, waiting for a user response. Upon receiving a response from the UE, the SN computes req_id using hash function and $Sign_{SN}$ by digital signature. Then, the SN puts req_id , $Sign_{SN}$ and $SUPI$ on the interface *bcChannel* and waits for the HN response.

Upon receiving the HN response, the SN picks $(xMac, HN_R)$ from the message and puts them on the interface *airChannel*, waiting for user response. In receiving the user response Res , the SN compares the Res with the $hxRes$ received from the HN. In the case of a match, the authentication is accepted, and accordingly the SN is able to retrieve K_{SEAF} and $SUPI$ from the HN response.

Similarly, the UE and HN processes, which are explained in Algorithm 7 and Algorithm 8, respectively, act and interact with the other processes. The main process is the starting point of the protocol. It initializes the protocol by setting up the key material, communication channels, and then invoking the participant process.

Algorithm 7 The User Process

```

let  $UE(ID_{HN}:\text{bitstring}, pk_{HN}:\text{publicKey}, K: UEKey, SUPI:\text{bitstring}) =$ 
in ( $airChannel, (R_1:\text{bitstring}, ID_{SN}:\text{bitstring})$ );
event  $UERecvSNReq(ID_{SN})$ ;
(*new  $R_2$ : bitstring;*)
let  $UI = pkiEnc((SUPI, R_1, R_2, ID_{SN}), pk_{HN})$  in
let  $SUCI = (UI, ID_{HN})$  in
event  $UESendResToSN(SUCI)$ ;
out ( $airChannel, SUCI$ );
in ( $airChannel, (xMac:\text{bitstring}, HN_R:\text{bitstring})$ );
let  $ue_{R_3} = symDec(HN_R, mKey(R_2))$  in
let  $O = f_1(K, R_1, (R_2, R_3))$  in
let  $Mac = f_1(K, O, ID_{SN})$  in
if  $Mac = xMac$  then event  $UERecvSNReq2(xMac)$ ;
let  $Res = challenge(K, O, ID_{SN})$  in
let  $ue_{K_{SEAF}} = keyseed(K, O, ID_{SN})$  in
event  $UESendRes2ToSN(Res)$ ;
out ( $airChannel, Res$ ).

```

As shown in Algorithm 9, the main process, which begins by the keyword *process*, generates the private asymmetric keys sk_{SN} and sk_{HN} for the principal SN and HN, respectively. The corresponding public keys are derived by invoking $getPK(sk_{SN})$ and $getPK(sk_{HN})$. Then, the results are revealed on the public interfaces *bcChannel* and *airChannel*, ensuring that the public keys are accessible by any attacker. Further, it generates the identifiers ID_{HN} and ID_{SN} respectively for the principals SN and HN.

Finally, the main process instantiates multiple copies of the HN, SN, and UE macros with the corresponding parameters serving as multiple sessions for each principal. It provides the benefits of multiple concurrent sessions for the attacker to investigate the possibility of interleaving attacks.

3) SECURITY PROPERTIES

ProVerif attempts to prove that a state in which a security property is violated is unreachable. Indeed, ProVerif can prove reachability properties, correspondence assertions, and observational equivalence. It also facilitates checking whether a specific term is available to an attacker; and accordingly, the secrecy of terms can be evaluated with respect to a model.

A security property is defined by the statement “query attacker(M)” to test the secrecy of term M in the model. Note that M is a ground term, which is likely private and not initially known to the attacker. We consider the following attacks:

- Attack on SUPI
- Attack on session key
- Attack on the random number of UE and HN
- Attack on USIM internal key
- Attack on subscriber information

We define six security properties: secrecy of $SUPI$, K_{SEAF} , R_2 , R_3 , and K . The secrecy of $SUPI$ aims to preserve the

Algorithm 8 The HN Process

```

let  $HN(ID_{HN}:\text{bitstring}, ID_{SN}:\text{bitstring}, pk_{HN}:\text{publicKey}, sk_{HN}:\text{privateKey}, pk_{SN}:\text{publicKey}, K: UEKey) =$ 
in ( $bcChannel, (req\_id:\text{bitstring}, sign_{SN}:\text{bitstring}, SUCI:\text{bitstring})$ );
let  $(=pk_{SN}, veri\_req\_id:\text{bitstring}) = checksign(sign_{SN}, pk_{SN})$  in

if  $veri\_req\_id = req\_id$  then event
 $HNRecvSNReq(req\_id, sign_{SN})$ ;
let  $UI = decode(SUCI)$  in
let  $dec\_UI = pkiDec(UI, sk_{HN})$  in
let  $HN\_R_2 = decode(dec\_UI)$  in
let  $R_1 = decode(dec\_UI)$  in
let  $HN\_SUPI = decode(dec\_UI)$  in
(*new  $R_3$ : bitstring; *)
let  $O = f_1(K, R_1, (HN\_R_2, R_3))$  in
let  $xMac = f_1(K, O, ID_{SN})$  in
let  $xRes = challenge(K, O, ID_{SN})$  in
let  $hxRes = hash((R_1, xRes))$  in
let  $HN\_K_{SEAF} = keyseed(K, O, ID_{SN})$  in
let  $EK = symEnc((HN\_K_{SEAF}, HN\_SUPI), mKey(xRes))$  in
let  $EK_C = pkiEnc(EK, pk_{SN})$  in
let  $HN\_R = symEnc(R_3, mKey(R_2))$  in
let  $res\_id = hash((hxRes, xMac, EK_C))$  in
let  $Sign_{HN} = sign((req\_id, res\_id), sk_{HN})$  in
event  $HNSendResToSN(req\_id, res\_id)$ ;
out ( $bcChannel, (EK_C, xMac, hxRes, Sign_{HN}, req\_id, res\_id, HN\_R)$ ).

```

privacy of the end user. Further, the secrecy of K_{SEAF} , R_3 , R_2 , and K presupposes the confidentiality of either pre-shared key (USIM internal key) or credential information.

In addition to secrecy, ProVerif can check authentication properties. Authentication can be captured by corresponding assertions which are used to capture relationships between events. This is expressed in the form of “if an event e has been executed, then event e' has been previously executed”. Annotating the process with events marks important stages reached by the protocol but which do not affect other behavior. In our protocol, we have six pairs of events:

- $(SNSendReqToUE, UERecvSNReq)$
- $(UESendResToSN, SNRecvUERes)$
- $(SNSendReqToHN, HNRecvSNReq)$
- $(HNSendResToSN, SNRecvHNRes)$
- $(SNSendReq2ToUE, UERecvSNReq2)$
- $(UESendRes2ToSN, SNRecvUERes2)$

The events in first pair indicate the sending of the request to the UE by the SN and the reception of this request by the UE. These events can be bound together by an inj-event statements. This is done by the first query statement in main process. This means that before event $UERecvSNReq$, event $SNSendReqToUE$ must be raised. This binding imposes an order on the sequence of message exchanged by the SN and the UE.

Algorithm 9 Security Properties and the Main Process

```

query attacker (SUPI).
query attacker (KSEAF).
query attacker (R3).
query attacker (R2).
query attacker (K).

query x:bitstring; event(UERecvSNReq(x)) ==>
    event(SNSendReqToUE).
query x:bitstring, y:bitstring; event(SNRecvUERes(x))
    ==> event(UESendResToSN(y)).
query x:bitstring, y:bitstring, z:bitstring;
    event(HNRecvSNReq(x,y)) ==>
    event(SNSendReqToHN(z)).
query x:bitstring, y:bitstring, z:bitstring;
    event(SNRecvHNRes(x)) ==>
    event(HNSendResToSN(y,z)).
query x:bitstring, y:bitstring; event(UERecvSNReq2(x))
    ==> event(SNSendReq2ToUE(y)).
query x:bitstring; event(SNRecvUERes2) ==>
    event(UESendRes2ToSN(x)).

process
new skHN:privateKey;
new skSN:privateKey;
new IDHN:bitstring;
new IDSN:bitstring;
let pkHN = pk(skHN) in out (airChannel, pkHN);
let pkSN = pk(skSN) in out (bcChannel, pkSN);
    (!SN(IDSN, IDHN, pkHN, pkSN, skSN)|
    !HN(IDHN, IDSN, pkHN, skHN, pkSN, K)|
    !UE(IDHN, pkHN, K, SUPI))

```

In a similar way, the other pairs of events are bound together with a query statement in the main process. ProVerif can check whether the bindings are satisfied during the execution of the protocol. Thus, reaching the state of the protocol to the point of *SNRecvUERes2* means that the required messages are exchanged by the participants. Further, the authentication is hold if the last message is successfully verified by the user.

D. VERIFICATION RESULTS

The results of each ProVerif query statement is presented by one of these arguments:

- RESULT [Query] is true: It means that the query is proved, and there is no attack.
- RESULT [Query] is false: It means that the query is false, and that ProVerif has discovered an attack against the specified security property.
- RESULT [Query] cannot be proved: This is a “don’t know” answer.

The verification results of the secrecy properties of the proposed protocol are summarized as follows:

- RESULT not attacker (*SUPI*[]) is **true**.
- RESULT not attacker (*K_{SEAF}*[]) is **true**.

- RESULT not attacker (*R₃*[]) is **true**.
- RESULT not attacker (*R₂*[]) is **true**.
- RESULT not attacker (*K* []) is **true**.

The above results show that the secrecy of *SUPI*, *K_{SEAF}*, *R₃*, *R₂*, and *K* are preserved by the protocol and there is no information leakage for these sensitive data.

Further, for the authentication properties of the proposed protocol, the verification results are as follows:

- RESULT event (*UERecvSNReq*(x)) ==> event (*SNSendReqToUE*) is **false**.
- RESULT event (*SNRecvUERes*(x)) ==> event (*UESendResToSN*(y)) is **true**.
- RESULT event (*HNRecvSNReq*(x,y)) ==> event (*SNSendReqToHN*(z)) is **true**.
- RESULT event (*SNRecvHNRes*(x)) ==> event (*HNSendResToSN*(y,z)) is **true**.
- RESULT event (*UERecvSNReq2*(x)) ==> event (*SNSendReq2ToUE*(y)) is **true**.
- RESULT event (*SNRecvUERes2*) ==> event (*UESendRes2ToSN*(x)) is **true**.

This indicates that all of the authentication properties, except the first one, are satisfied by the protocol.

By tracing the verification result of the first message, we can see that any attacker can act as a SN by generating a correct message for the first phase of the protocol. This is because the attacker can pick up a random *R₁* and send it along with the *ID_{SN}* to the subscriber. It should be noted that there are no shared secrets between the subscriber and the SN to enable the integrity of the first message of the protocol. The only option for enabling the authentication of the SN in the first message is that the SN adds its digital signature to the request. This solution, which requires a signature verification from the subscriber, is still vulnerable to a replay attack, since the attacker can eavesdrop on the radio interface and collect SN authentication requests sent to subscriber. The attacker can pick up one of these requests and send it to the intended subscriber. Thus, as in 5G AKA, we ignore this security property in our protocol.

Note that except for the first phase of the protocol, the attacker cannot violate the authentication properties of the other phases. The message between the SN and HN are protected by the digital signature of the sender as well as the public key encryption. Further, the integrity of the messages between the UE and HN is protected by the pre-shared key *K* and public key encryption.

V. EVALUATIONS

In this section, we evaluate the performance of the proposed protocol. We begin by explaining the implementation of smart contract functions. Next, to provide experimental results, we use the Ethereum blockchain [23], where the block generation time is approximately 12 seconds. We then present an analysis of the smart contract functions in terms of execution and transaction costs. Finally, we compare the proposed protocol with well-known prior work.

A. SMART CONTRACT FUNCTIONS

Since, the smart contract acts as an interface between SN and HN, it has two functions that facilitate a bidirectional channel for exchanging messages between SN and HN. The operations of these functions are respectively corresponding to the phase 4 and phase 6 of the protocol. Algorithm 10 shows the implementation of these functions in Solidity, an object-oriented, high-level language for implementing smart contracts [24]. Further, there is a data structure, namely *AuthToken*, for storing the information of authentication requests and responses. The function *SetSNRequest* is responsible for processing authentication request while *SetHNResponse* handles the authentication response. The former receives the SN request, including *req_id*, *sign_{SN}*, *ID_{SN}*, and *SUCI* as inputs, creates an *AuthToken* object and initializes it by the input parameters. Then it puts the new object into a data map structure where the *req_id* acts as the key of the map. Finally, it receives the access fee in GAS.

The latter is invoked by the HN to register the response of the authentication request. It searches the map for an *AuthToken* object whose key is equal to the input parameter *req_id*. It sets the value of corresponding fields if the target object is found. Otherwise, it returns an error and the transaction is aborted.

Note that the first line in *SetHNResponse* implies that the sender of the transaction can only be the owner of the smart contract. That is, anyone other than the HN is unable to register a transaction whose destination is the address of *SetHNResponse*. This restriction is another security feature which prohibits an attacker from acting as the HN and registering a response transaction.

B. PERFORMANCE ANALYSIS

The computing energy required to process and validate transactions on the Ethereum blockchain is measured by means of gas. A higher gas limit means more work to execute a transaction or a smart contract. The gas is paid by the end user who registers the transaction to allocate resources of the Ethereum virtual machine in order that decentralized applications such as smart contracts can self-execute in a secured manner. Gas prices are denoted in small fractions of ether called *gwei*. One ether is equal to 10^9 *gwei*.

Table 2 shows the results of the smart contract implementation on the Ethereum blockchain. Here, we focus on both *SetSNRequest* and *SetHNResponse* functions. The cost of each function depends on both the message size and instructions executed. As shown in Table 2, the cost is paid for both transaction registration and function execution.

To evaluate different security levels, we consider two specific hash functions: SHA-1 with 20-byte output and SHA-2 with 32-byte output. For symmetric encryption, we use AES with a 128-bit key. The output of each encryption block is 16 bytes. For identifiers such as *ID_{SN}* and *SUCI*, we assume a binary string of 16 bytes in length.

Algorithm 10 Solidity Code for Smart Contract Functions

```
pragma solidity ^0.5.1;
contract AuthenticationContract
{
    address owner;
    constructor() public
    {
        owner = msg.sender;
    }
    struct AuthToken{
        string res_id;
        string EK;
        string xMac;
        string hxRes;
        string signHN;
        string HN_R;
        string signSN;
        string IDSN;
        string SUCI;
        bool active;
    }
    mapping(string => AuthToken) private MAP;
    function SetSNRequest (
        string memory req_id,
        string memory signSN,
        string memory IDSN,
        string memory SUCI)
        public payable returns (string memory)
    {
        if (MAP[req_id].active)
            return "Error!";
        MAP[req_id] = AuthToken ({signSN, IDSN, SUCI,
            true, EK:"", xMac:"", hxRes:"", signHN:"", res_id:"",
            HN_R:""});
        return "Done";
    }
    function SetHNResponse (
        string memory EK,
        string memory xMac,
        string memory hxRes,
        string memory signHN,
        string memory req_id,
        string memory res_id,
        string memory HN_R)
        public returns(string memory)
    {
        require (msg.sender==owner);
        AuthToken memory Tok = MAP[req_id];
        if (Tok == NULL)
            return "Error";
        Tok.EK = EK;
        Tok.xMac = xMac;
        Tok.hxRes = hxRes;
        Tok.signHN = signHN;
        Tok.res_id = res_id;
        Tok.HN_R = HN_R;
        Tok.active = false;
        return "Done";
    }
}
```

TABLE 2. Transaction and execution costs of smart contract functions (1 gas = 10^{-9} ETH).

Functions	Hash	Execution Cost (gas)	Transaction Cost (gas)
<i>SetSNRequest</i>	SHA-1	493010	552618
	SHA-2	533350	600382
<i>SetHNResponse</i>	SHA-1	309068	355940
	SHA-2	309438	358678

We can see that the hash algorithm does not have a significant impact on the cost paid for *SetHNResponse* function. But, in the *SetSNRequest* function, the transaction and execution costs of SHA-2 is about 10% higher than the costs for SHA-1.

C. COMPARISON TO PRIOR WORK

Table 3 presents a brief comparison of our proposed protocol to well-known previous protocols for authentication, including 4G-AKA, 5G-AKA, and Braeken's protocol [11]. The 4G-AKA (also referred as EPS-AKA²) has two weaknesses:

- The UE always sends its permanent identifier in clear text to the network, allowing it to be stolen by either a malicious network or a passive adversary over the radio interface;
- The HN is consulted during authentication only to generate authentication vectors; it does not make decisions on the authentication results.

TABLE 3. Comparison with other protocols.

	This work	[11]	5G-AKA	4G-AKA
Sending the UE identity in cipher text	✓	✓	✓	✗
Making a final decision of authentication by the HN	✓	✓	✓	✗
Prohibiting subscribers from tracking due to <i>MAC</i> or synchronization failure	✓	✓	✗	-
Prevention of information leakage from <i>SQN</i> parameter	✓	✓	✗	-
Protection against malicious SN	✓	✓	✗	-
Preserving user privacy when the SN is compromised	✓	✓	✗	-
Unlinkability against active attacker	✓	✓	✗	✗
Blockchain-based	✓	✗	✗	✗
Distributed (DoS prevention)	✓	✗	✗	✗
Auditable log	✓	✗	✗	✗
Low latency	✗	✓	✓	✓
Free of secure channel assumption between HN and SN	✓	✗	✗	✓
Supporting digital currency	✓	✗	✗	✗

Both the mentioned weaknesses have been resolved by 5G-AKA. First, the UE uses the public key of the HN and

encrypts the permanent identifier before it is sent over the radio interface. Second, the HN (i.e., the AUSF) makes the final decision on the UE authentication. Finally, results of authentication are sent to the UDM to be logged.

However, 5G-AKA, as currently stated in Sec. II, has the following vulnerabilities:

- Information leakage from *SQN* parameter;
- Tracking of targeted subscribers due to different failure in *MAC* or Synchronization;
- Impersonation of a malicious SN as a legitimate SN;
- Compromising of an SN by an attacker can lead to a complete breach of user privacy.

The *SQN* mechanism was mainly designed for replay protection in 3G when the USIM is too limited to produce good pseudo random numbers. Nowadays, this is no longer true in 5G, since USIMs can generate good pseudo random numbers in addition to handling asymmetric encryption. Thus, our protocol, similar to [11], uses a standard challenge-response mechanism to replace the *SQN* counters. More specifically, all participants, namely the SN, UE and HN, generate the random numbers R_1 , R_2 , and R_3 , respectively, which involve a challenge-response mechanism as well as a derivation of the session key. The elimination of *SQN* also avoids any de-synchronization attacks between the user and HN.

The last two weaknesses are eliminated by the modification in message encryption. First, the UE verifies the validity of the authentication response received from the SN. Second, the SN which receives an encrypted message, composed of a *SUPI* and session key, is only capable of decrypting the message when it satisfies two conditions: 1) owning the private key of the SN and 2) reception of a commitment from the UE, by *Res* parameter, which approves the authentication response.

We should point out that our protocol, by virtue of existing on a blockchain, has some odd features. It is inherently distributed, thus DoS attacks against the HN cannot occur during the running of the protocol. Further, it is auditable, and the legitimate nodes, including both the HN and SN, can obtain information on authentication requests and responses by scanning the blockchain transactions and decrypting the messages. As a result, undeniability of both requests and responses are satisfied by the proposed protocol, while this is not supported by either 5G-AKA or Braeken's protocol.

However, using blockchain has some side effects. The major drawback of blockchain is the transaction delay, which imposes an inevitable latency on the whole system. For example, on the Ethereum platform, the transaction delay is about 12 seconds. Since we need two transactions at every run of the protocol, the total delay is 25 seconds. Although this transaction delay is considerable compared to non-blockchain AKA protocols, it is a reasonable trade-off when we consider the issues not supported by the other protocols. Moreover, using a private blockchain, such as Hyperledger, can reduce the transaction time by restricting the set of full nodes. In this solution, each HN sets up its own blockchain network with a small set of full nodes to run the consensus algorithm.

²Evolved Packet System Authentication and Key Agreement

Unlinkability against an active attacker is another feature that is satisfied by our protocol. As the authors mentioned in [24], linkability attacks can be seen in current LTE protocol and earlier versions of it. Further, Basin's privacy analysis shows that the 5G AKA fails to ensure unlinkability against an active attacker [9]. In our protocol, unlinkability against active attacker is originated from the confidentiality of the *SUPI* which only an authenticated SN can reveal it at the end of a successful run. Otherwise, *SUPI* cannot be exposed to a malicious SN.

Furthermore, the proposed protocol is far from the assumption for secure channel between the HN and SN which is a primary requirement of the previous protocol and many vulnerabilities may be ignored by this assumption. This feature is the result of encrypting the exchanged messages which are located in the metadata of transactions on the public blockchain. From a practical point of view, there is no need for the HN to establish a secure session with the SN for message exchanging.

Finally, the public nature of the blockchain provides an opportunity to use digital currency for service payments. In this scenario, the UE, which receives service from the SN, can pay for the cost of service directly to the SN, thereby avoiding an otherwise complex financial transaction process and cleaning room between the HN and SN.

VI. APPLICATION TO AERIAL NETWORKS

Various forms of aerial base stations have started receiving substantial attention in a wide range of applications since they have the potential of providing enhanced and seamless connectivity in 5G and beyond (B5G) networks. Due to their 3D-mobility and adaptable altitude, aerial base stations can efficiently support the connectivity of both terrestrial and aerial users (such as cargo drones). Based on the operation altitudes, two main types of aerial base stations are distinguished, the ones deployed on Unmanned Aerial Vehicles (UAVs) and those in the form of High-Altitude Platform Station (HAPS) systems. UAVs operate at low altitudes of around a few hundred meters and act as flexible and agile nodes in the form of mere relays or fully fledged base stations (UAV-BSs, referred to as UxNB in recent 3GPP documents such as TR 22.829); in contrast, HAPS systems operate at higher altitudes of 8 to 50 km above the ground (TR 38.811). It is envisioned that HAPS systems will facilitate 3D highways which may drive a disruptive transformation in the retail industry [25]. HAPS systems also allow wider coverage areas and longer flight times compared to UAV-BSs. In practice, HAPS can act as a super macro base station to provide connectivity in a plethora of applications [26].

By means of UAV-BSs and HAPS systems, we can set up local SNs to provide roaming service in different situations. Roaming will have a totally new meaning in the envisioned B5G multi-tier network; this paradigm is referred to as a *Vertical Heterogenous Network (VHetNet)* in the recent literature [27]. This is a novel wireless access architecture in the form of a network of networks owned and managed

by different operators including terrestrial, aerial, and even satellite. Our scheme can facilitate the authentication of users in such sophisticated B5G networks as outlined below:

- It has a decentralized and fault tolerant architecture which is secure against DoS, DDoS and replay attacks,
- It is highly protected against malicious SNs which would have likely appeared in *VHetNet* framework,
- By means of blockchain it provides a platform to connect different type of operators,
- It provides an auditable log for tracking abnormal activities.

VII. CONCLUSION

We proposed a novel AKA protocol for roaming service in 5G mobile networks. The protocol uses blockchain to communicate messages between the HN and SN. The protocol was designed in order to leverage the benefits of blockchain, including its undeniable and auditable logs and its preservation of privacy. Further, the protocol eliminates the need for a secure communication channel between the HN and SN. We proved the security of the protocol with a formal verification procedure. The protocol was modeled in ProVerif and successfully verified the secrecy of sensitive data as well as the authentication of the participants. The evaluation results on the Ethereum blockchain showed that the cost of implementation in GAS are acceptable.

REFERENCES

- [1] D. Fang, Y. Qian, and R. Q. Hu, "Security for 5G mobile wireless networks," *IEEE Access*, vol. 6, pp. 4850–4874, 2018.
- [2] *Security Architecture and Procedures for 5G System*, document TS 33.501, 3GPP, May 2019.
- [3] P. Ahokangas, M. Matimikko-Blue, S. Yrjölä, V. Seppänen, H. Hämmäinen, R. Jurva, and M. Latva-Aho, "Business models for local 5G micro operators," *IEEE Trans. Cognit. Commun. Netw.*, vol. 5, no. 3, pp. 730–740, Sep. 2019.
- [4] Y. Siriwardhana, P. Porambara, M. Liyanage, J. S. Walia, M. Matimikko-Blue, and M. Ylianttila, "Micro-operator driven local 5G network architecture for industrial Internet," in *Proc. IEEE Wireless Commun. Netw. Conf. (WCNC)*, Apr. 2019, pp. 1–8.
- [5] P. Hegedüs, "Towards analyzing the complexity landscape of solidity based ethereum smart contracts," *Technologies*, vol. 7, no. 1, p. 6, Jan. 2019.
- [6] *Internet Key Exchange Protocol Version 2 (IKEv2)*, document RFC 7296, IETF, 2014. [Online]. Available: <https://tools.ietf.org/html/rfc7296>
- [7] R. Borgaonkar, H. Lucca, P. Shinjo, and S. Altaf, "New privacy threat on 3G, 4G, and upcoming 5G AKA protocols," in *Proc. Privacy Enhancing Technol.*, no. 3, 2019, pp. 108–127.
- [8] M. Liyanage, J. Salo, A. Braeken, T. Kumar, S. Seneviratne, and M. Ylianttila, "5G Privacy: Scenarios and Solutions," in *Proc. IEEE 5G World Forum*, Jul. 2018, pp. 197–203.
- [9] D. Basin, J. Dreier, L. Hirschi, S. Radomirovic, R. Sasse, and V. Stettler, "A formal analysis of 5G authentication," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, Jan. 2018, pp. 1383–1396.
- [10] S. Meier, B. Schmidt, C. Cremers, and D. Basin, "The TAMARIN prover for the symbolic analysis of security protocols," in *Proc. Int. Conf. Comput. Aided Verification*, Saint Petersburg, Russia, 2013, pp. 696–701.
- [11] A. Braeken, M. Liyanage, P. Kumar, and J. Murphy, "Novel 5G authentication protocol to improve the resistance against active attacks and malicious serving networks," *IEEE Access*, vol. 7, pp. 64040–64052, 2019.
- [12] M. Tahir, M. H. Habaebi, M. Dabbagh, A. Mughees, A. Ahad, and K. I. Ahmed, "A review on application of blockchain in 5G and beyond networks: Taxonomy, field-trials, challenges and opportunities," *IEEE Access*, vol. 8, pp. 115876–115904, 2020.

- [13] A. Refaey, K. Hammad, S. Magierowski, and E. Hossain, "A blockchain policy and charging control framework for roaming in cellular networks," *IEEE Netw.*, vol. 34, no. 3, pp. 170–177, May 2020.
- [14] A. V. Rivera, A. Refaey, and E. Hossain, "A blockchain framework for secure task sharing in multi-access edge computing," *IEEE Netw.*, early access, Sep. 30, 2020, doi: [10.1109/MNET.011.2000497](https://doi.org/10.1109/MNET.011.2000497).
- [15] Z. Chen, S. Chen, H. Xu, and B. Hu, "A security authentication scheme of 5G ultra-dense network based on block chain," *IEEE Access*, vol. 6, pp. 55372–55379, 2018.
- [16] (Dec. 2017). *BubbleTone Blockchain*. [Online]. Available: <https://coinwoot.com/wp-content/uploads/BubbleTone-Yellow-Paper.pdf>.
- [17] A. Idris, A. Khan, and Y. S. Lee, "Genetic programming and adaboosting based churn prediction for telecom," in *Proc. IEEE Int. Conf. Syst., Man, Cybern. (SMC)*, Oct. 2012, pp. 1328–1332.
- [18] *Study on New Radio (NR) Access Technology Physical Layer Aspects*, document TR 38.802, 3GPP, Mar. 2017.
- [19] Y. Zhang, S. He, S. Li, and J. Chen, "Intra-operator customer churn in telecommunications: A systematic perspective," *IEEE Trans. Veh. Technol.*, vol. 69, no. 1, pp. 948–957, Jan. 2020.
- [20] *Digital Cellular Telecommunications System (Phase 2+) (GSM); Universal Mobile Telecommunications System (UMTS); 3G Security; Security Architecture*, document ETSI, TS 33.102 version 16.0.0 Release, 3GPP, Aug. 2020.
- [21] M. Liyanage, P. Kumar, M. Ylianttila, and A. Gurtov, "Novel secure VPN architectures for LTE backhaul networks," *Secur. Commun. Netw.*, vol. 9, no. 10, pp. 1198–1215, Jul. 2016.
- [22] B. Blanchet, M. Abadi, and C. Fournet, "Automated verification of selected equivalences for security protocols," *J. Log. Algebr. Program.*, vol. 75, no. 1, pp. 3–51, Feb. 2008.
- [23] G. Wood. (Sep. 5, 2020). *Ethereum: A Secure Decentralized Generalized Transaction Ledger; Petersburg Version 3e2c089*. [Online]. Available: <https://ethereum.github.io/yellowpaper/paper.pdf>
- [24] C. Dannen, *Introducing Ethereum and Solidity*, vol. 1. Berkeley, CA, USA: Apress, 2017.
- [25] N. Cherif, W. Jaafar, H. Yanikomeroglu, and A. Yongacoglu, "3D aerial highway: The key enabler of the retail industry transformation," 2020, *arXiv:2009.09477*. [Online]. Available: <http://arxiv.org/abs/2009.09477>
- [26] M. S. Alam, G. K. Kurt, H. Yanikomeroglu, P. Zhu, and N. D. Dao, "High altitude platform station based super macro base station (HAPS-SMBS) constellations," 2020, *arXiv:2007.08747*. [Online]. Available: <http://arxiv.org/abs/2007.08747>
- [27] T. Darwish, G. K. Kurt, H. Yanikomeroglu, G. Senarath, and P. Zhu, "A vision of self-evolving network management for future intelligent vertical HetNet," 2020, *arXiv:2009.02771*. [Online]. Available: <http://arxiv.org/abs/2009.02771>



MAEDE HOJJATI was born in Tehran, Iran, in 1993. She received the B.S. degree in computer engineering from Arak University, in 2015, and the M.Sc. degree in computer engineering from Tarbiat Modares University, Tehran, in 2020. Since 2019, she has been a Researcher with the Security Evaluation Laboratory (SE-Lab), Tarbiat Modares University. She is currently working as a Penetration Testing. Her research interests include network security, 5G networks, and blockchain.

Her current research interests include security and privacy protocols for IoT, blockchain, and 5G security.



ALIREZA SHAFIEINEJAD received the B.S. degree in computer engineering from the Sharif University of Technology, Tehran, in 1999, and the M.Sc. and Ph.D. degrees in electrical engineering from the Isfahan University of Technology, Isfahan, Iran, in 2002 and 2013, respectively. Since 2015, he has been an Assistant Professor with Tarbiat Modares University, Tehran, Iran. He is currently an Assistant Professor with the Department of Electrical and Computer Engineering, Tarbiat Modares University. His research interests include wireless network coding, network security, and design and analysis of cryptography algorithms. At Tarbiat Modares University, he leads research in network security and penetration test in the Security Evaluation Laboratory (SE-Lab).



HALIM YANIKOMEROGLU (Fellow, IEEE) is currently a Professor with the Department of Systems and Computer Engineering, Carleton University, Ottawa, ON, Canada. His current research interest includes many aspects of 5G/6G wireless networks. His collaborative research with industry has resulted in 37 granted patents. He is also a Fellow of the Engineering Institute of Canada (EIC) and the Canadian Academy of Engineering (CAE). He is also a Distinguished Speaker of IEEE Communications Society and IEEE Vehicular Technology Society. He received several awards for his research, teaching, and service, including the IEEE Communications Society Wireless Communications Technical Committee Recognition Award in 2018 and the IEEE Vehicular Technology Society Stuart Meyer Memorial Award in 2020. He is also serving as the Chair for the IEEE Wireless Communications and Networking Conference (WCNC) Steering Committee. He was the Technical Program Chair/Co-Chair of WCNC 2004, Atlanta, WCNC 2008, Las Vegas, and WCNC 2014, Istanbul. He was the General Chair of IEEE VTC 2010-Fall, Ottawa, and VTC 2017-Fall, Toronto. He has also served as the Chair for the IEEE's Technical Committee on Personal Communications.

...